LetLearnTogether

Frontend Documentation Pack

Architecture, routes, screens, API integration map, PWA and low-bandwidth plan, design system and delivery plan

Version	1.0 (draft for design review)
Date	27 October 2026
Project	Open Student Past Papers & Library Platform (System Design course)
Prepared by	Frontend Team
Based on	PRD v3.0, SRS v3.0, backend docs (API contracts, architecture, integration, coordination, code review) and the draft API_SPEC.yaml received
Final submission	Tuesday 17 November 2026

Contents

1. Purpose and scope
2. Technology choices
3. Frontend architecture diagram
4. Project structure
5. Route map
6. Role access matrix
7. Screen-to-endpoint map
8. API integration rules
9. State and data handling
10. PWA and low-bandwidth plan
11. Design system and accessibility
12. Mobile wireframes (low fidelity)
13. Delivery plan and responsibilities
14. Testing and definition of done
15. Contract alignment and open questions

How to read the status labels

Label	Meaning
In YAML	The route appears in the draft API_SPEC.yaml received on 27 Oct 2026.
Docs only	The route is described in the backend docs (api contracts, integration) but is not in the YAML we received.
Conflict	The YAML and the backend docs disagree. The frontend follows the assumption written beside it until the backend lead confirms.
Pending	Not defined anywhere yet. The frontend needs a decision before building that part.

Important: the YAML we received appears to be an early draft (about 8 operations) while the backend docs call API_SPEC.yaml authoritative and list roughly twice as many routes. This pack is built against the routes in the backend docs and marks everything the YAML does not confirm. When the current YAML arrives, section 7 and section 15 are the only places that need updating.

1. Purpose and scope

This pack describes how the LetLearnTogether frontend is designed and built. It is the frontend section of the System Design Document and the working reference for the four frontend team members.

What the frontend delivers

- A mobile-first React web app that works as an installable Progressive Web App (PWA) from the first deployment.
- The four product areas from the PRD: the document library (folder-style browsing), verification and digital stamp page, quizzes, and the screens that support them (accounts, upload status, review, moderation, administration).
- No business logic in the frontend. Approval rules, scoring and permissions are enforced by the API; the UI only reflects them.

Course deliverables owned by the frontend / UX role

- Wireframes, mobile-friendly UI, installable PWA (manifest and service worker).
- Accessibility and low-bandwidth checks, integration with the API, and the user manual with screenshots (installing the PWA, uploading and verifying a paper, taking a quiz, checking a stamp, troubleshooting).
- Week 1 requirement: interviews with at least five classmates and written findings. Findings should be linked from the System Design Document.

Source documents

Document	Used for
PRD v3.0 / SRS v3.0	Roles, folder structure, functional and non-functional requirements, acceptance criteria
Backend docs (5 files)	API conventions, authorization rules, error format, upload rules, local setup and CORS
API_SPEC.yaml (draft received)	Request and response shapes for auth, documents, verification, quizzes
Course concept paper	Deadlines, required features, PWA and low-bandwidth rules, grading

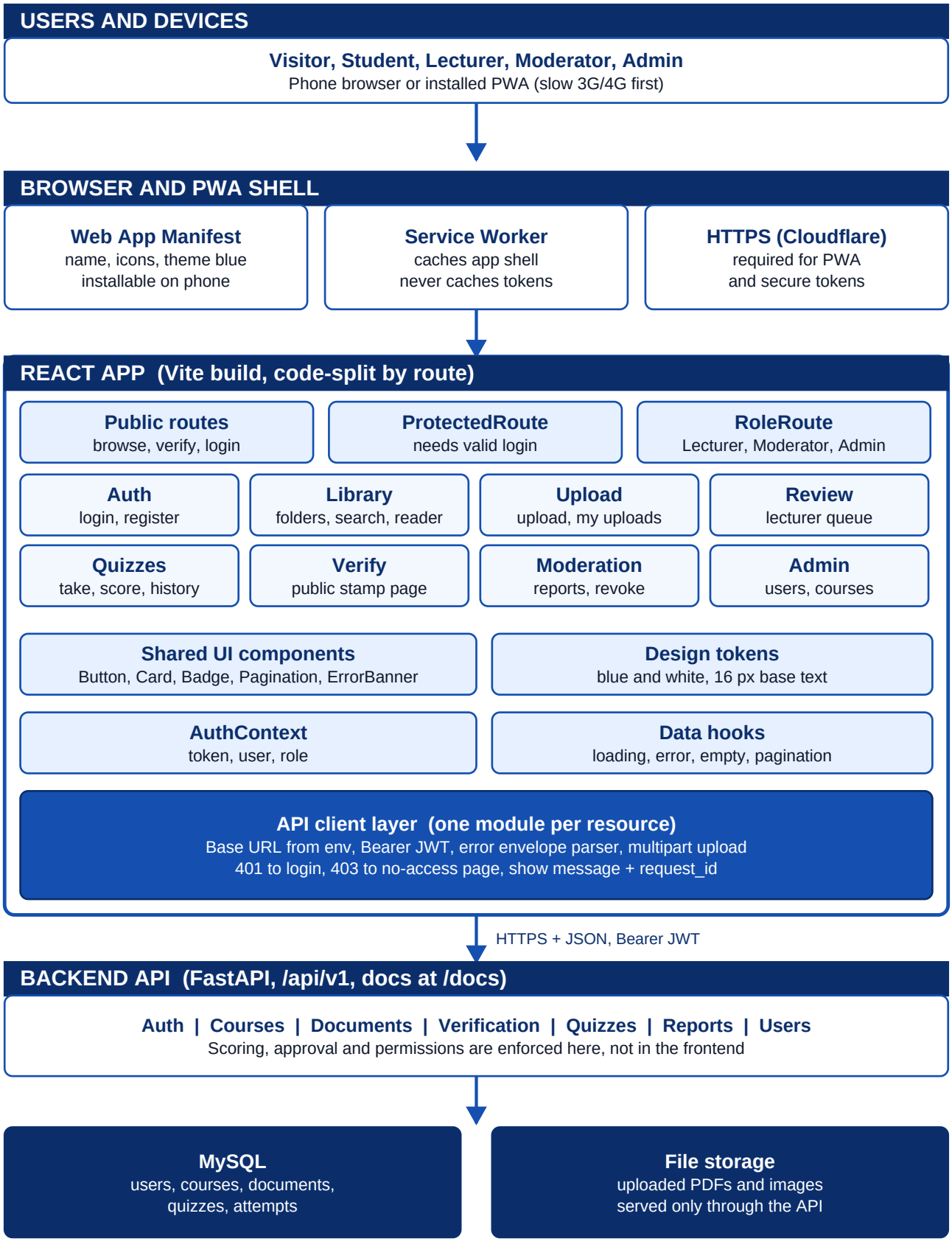
2. Technology choices

The PRD fixes React + Vite for the frontend. Items marked *Recommended* are the frontend team's proposals and need the lead's confirmation.

Area	Choice	Source	Why
Framework / build	React + Vite	PRD	Fast builds, component model, good PWA tooling
Routing	React Router (nested routes)	Recommended	SRS requires nested routes such as /courses/CS101/2025/First-Semester/Midterm
Server data	Small fetch wrapper + TanStack Query	Recommended	Caching, loading and error states, pagination with little code
Global state	React Context for auth only	Recommended	Keeps state simple; everything else comes from the API
PWA	vite-plugin-pwa (Workbox)	Recommended	Generates manifest and service worker; fewer hand-written parts to break
Styling	Plain CSS with design tokens (CSS variables)	Recommended	No heavy UI kit, smaller bundle on slow networks
Testing	Vitest + React Testing Library; MSW for API mocks	Recommended	MSW can mock endpoints from the OpenAPI contract so work continues if the backend is late
Quality	ESLint + Prettier; CI runs lint and tests on every pull request	Course rule	Concept paper requires CI on each PR
Hosting	Static build served by Nginx on the VPS behind Cloudflare HTTPS	PRD (Nginx, Cloudflare)	OPS to confirm the exact deploy path

AI-assisted coding is allowed by the course. Every pull request must state where AI helped, and each member must be able to explain their code at the oral defence.

3. Frontend architecture diagram



Font sizes used in this diagram: layer headings 11 pt bold, component names 10 pt bold, details 8.5 pt, connector labels 8 pt.

How a request moves through the frontend

- The user opens a route. **ProtectedRoute** checks AuthContext for a valid session; **RoleRoute** checks the role. If either fails the user goes to login or a no-access page.
- The page calls a data hook, which calls the resource module in the API client layer (for example documents or quizzes).
- The client adds the `Authorization: Bearer` header, sends JSON (or multipart for uploads), and reads the standard error envelope on failure.
- The hook returns loading, error, empty or success to the page. Every list screen must render all four states.
- The server decides what the user may see. The UI hides buttons a role cannot use, but never relies on hiding for security.

Layer responsibilities

Layer	Owns	Must not
Routes	URL structure, login and role guards	Contain data fetching or styling
Feature pages	Screens and their local form state	Call fetch directly; use the API client
Shared components	Reusable UI, accessibility behaviour	Know about API routes
API client	URLs, headers, error parsing, pagination helpers	Hold UI state
PWA shell	Install, app-shell caching	Cache tokens, uploads or personal data

4. Project structure

Proposed layout of the React repository folder. Names can change, but the separation of routes, features, components and API client should stay.

```
frontend/
  public/            manifest icons (192, 512), favicon
  src/
    api/             client.js (fetch wrapper), auth.js, courses.js,
                     documents.js, verification.js, quizzes.js, reports.js
    context/         AuthContext.jsx
    routes/          AppRoutes.jsx, ProtectedRoute.jsx, RoleRoute.jsx
    features/
      auth/          LoginPage, RegisterPage
      library/       CourseList, FolderBrowser, SearchPage, DocumentReader
      upload/        UploadPage, MyUploadsPage
      review/        ReviewQueuePage
      quizzes/       QuizList, TakeQuiz, ResultPage, AttemptHistory,
                     QuizEditor (lecturer)
      verify/        VerifyPage (public)
      moderation/    ReportsPage
      admin/         UsersPage, CoursesPage
    components/      Button, Input, Card, Badge, Pagination, Skeleton,
                     ErrorBanner, EmptyState, FileCard, FolderRow
    hooks/           useAuth, usePagedQuery
    styles/          tokens.css, base.css
    utils/           compressImage, formatDate, errorMessages
    tests/
  vite.config.js     includes vite-plugin-pwa settings
  .env.example       VITE_API_BASE_URL=http://localhost:8000/api/v1
```

5. Route map

Routes follow the SRS nested-route rule. The fourth folder level (lecturer) is in the PRD but not in the SRS route or query parameters, so it is shown as a filter on the type page until confirmed (see section 15).

Path	Screen	Access
/	Home: search box, course list, sign-in prompt	Public
/login, /register	Sign in, create Student account	Public
/courses	Level 1: list of courses	Public
/courses/:code/:year/:semester	Levels 2-3: document types for that term	Public
/courses/:code/:year/:semester/:type	Level 4: documents (filter by lecturer)	Public
/search	Keyword and filter search with pagination	Public
/documents/:id	Document detail and reader (view inline, download, report)	Public for verified documents; report needs login
/verify/:token	Public verification page (Verified or Revoked)	Public, no login
/quizzes	Published quizzes by course	Public list
/quizzes/:id	Take a quiz	Student
/quizzes/:id/result	Score and answer review	Student
/my/uploads	My uploads with review status	Logged in
/my/attempts	Quiz score history	Student
/upload	Upload a document	Logged in
/lecturer/review	Pending documents for my courses	Lecturer, Moderator, Admin
/lecturer/quizzes (and /new)	Create and publish quizzes	Assigned Lecturer, Admin
/moderation/reports	Review reports, revoke or remove content	Moderator, Admin
/admin/users, /admin/courses	Manage users, roles and courses	Admin
*	Not found page	Public

6. Role access matrix

What each role sees in the UI. The server enforces the same rules; the frontend hides actions a role cannot perform so users do not hit avoidable errors.

Action	Visitor	Student	Lecturer	Moderator	Admin
Browse folders and search	Yes	Yes	Yes	Yes	Yes
Read / download verified document	Yes*	Yes	Yes	Yes	Yes
Open verification page	Yes	Yes	Yes	Yes	Yes
Register / log in	Yes	-	-	-	-
Upload a document (creates a review request)	-	Yes	Yes	Yes	Yes
See my own uploads and status	-	Yes	Yes	Yes	Yes
Report a document	-	Yes	Yes	Yes	Yes
Take a quiz and see history	-	Yes	-	-	-
Create or edit quizzes	-	-	Own courses	-	Yes
Approve or reject a document	-	-	Own courses	Yes	Yes
Revoke stamp / remove content	-	-	-	Yes	Yes
Handle reports	-	-	-	Yes	Yes
Manage users, roles, courses	-	-	-	-	Yes

* The backend contract says verified library reads are public. The SRS lists reading and downloading under Student permissions. The frontend will allow public reading, as the backend states, until the team decides otherwise (section 15). Quiz attempts are Student-only in the backend contract, so the Start button is hidden for every other role.

7. Screen-to-endpoint map

Paths are relative to `/api/v1`. Status labels are explained on page 2.

Screen / action	Endpoint	Status	Frontend note
Register	POST /auth/register	In YAML	Send name, email, password only. YAML lists a role field; never send it.
Login	POST /auth/login	In YAML	Returns access_token and token_type.
Who am I / role	GET /auth/me	Docs only	Login response has no user data, so call this after login to get the role.
Course list (folder level 1)	GET /courses, /courses/{id}	Docs only	Also fills the course picker on upload.
Browse and search documents	GET /documents	In YAML	YAML filters: course_id, year, semester, document_type. Keyword, status, lecturer, page, page_size are Pending.
Document detail	GET /documents/{id}	Docs only	Response fields not yet seen.
Read inline / download	GET /documents/{id}/content?disposition=inline|attachment	Docs only	Fall back to download when the phone cannot show PDFs inline.
Upload	POST /documents (multipart)	In YAML	Fields: file, course_id, document_type. Year and semester: Pending.
My uploads and status	GET /documents/mine	Docs only	Shows Under Review, Verified, Rejected.
Review queue (list)	not defined	Pending	Need a way to list pending documents for a lecturer's courses.
Approve / reject	PUT /documents/{id}/review	In YAML	Body: action approve or reject.
Revoke stamp	PUT review action revoke (YAML) or separate revoke route (docs)	Conflict	Wrap in one function in api/documents.js so a change touches one line.
Verification page	GET /verification/{token}	In YAML	YAML: status, course_code, approval_date. SRS also needs document ID.
QR image	GET /verification/{token}/qr	Docs only	Shown on the stamp. QR must open the frontend /verify page.
Report a document	document report route	Docs only	Exact path not stated.
Moderator reports	GET /reports, /reports/{id}	Docs only	Management is Moderator or Admin.
Quiz list	GET /quizzes	In YAML	Published quizzes only.
Quiz detail (questions)	GET /quizzes/{id}	Docs only	Needed for question IDs. Never contains answers.
Create quiz	POST /quizzes	In YAML	Lecturer; correct_answer_index is sent only here.
Submit attempt	POST /quizzes/{id}/attempt (YAML) or /attempts (docs)	Conflict	Assume /attempts from the docs; isolate in api/quizzes.js.
Score history	GET /quiz-attempts/mine	Docs only	Student only.
Admin users / courses	GET /users, /courses (POST, PUT, DELETE)	Docs only	Admin only.

8. API integration rules

- **Base URL:** read from `VITE_API_BASE_URL`. Local value is http://localhost:8000/api/v1. The backend allows the Vite dev server on port 5173; production needs the real frontend domain added to its allowed origins (ask OPS and the backend lead).
- **Authentication:** send `Authorization: Bearer <token>` on protected calls. No refresh token exists in the docs, so an expired token means a 401 and the user is sent to login with their previous path remembered.
- **Errors:** every error is `{ error_code, message, request_id }`. Show the message to the user in plain words, keep request_id in a small "Details" line so students can quote it when reporting a problem, and map known error_code values to friendlier text in utils/errorMessages.
- **Status codes:** 401 to login; 403 to a no-access page; 404 to a not-found state; 413 or a size error to "file is too large (limit 10 MiB)"; 422 or 400 to field-level messages; network failure to a retry banner.
- **Lists:** responses are `{ page, page_size, total_items, total_pages, items }`. Pages start at 1, default size 20, maximum 100. Use a shared Pagination component and a usePagedQuery hook.
- **Upload:** multipart with file, course_id, document_type. Allowed types: PDF, JPEG, PNG, WebP; limit 10 MiB. Check type and size in the browser first and compress photos before sending. HEIC photos from iPhones are not allowed, so convert them or show a clear message. A new upload returns Under Review and is not public until approved.
- **Viewing files:** use the content route with disposition=inline to view and disposition=attachment to download. Never construct file paths in the UI.
- **Quizzes:** send answers as question ID plus selected option index. Never send or compute a score on the client. Quiz reads never include correct answers.
- **One place for routes:** all URLs live in src/api. When a route changes, only that file changes.

9. State and data handling

Kind of state	Where it lives	Notes
Session (token, user, role)	AuthContext	Loaded at start; user fetched with GET /auth/me after login
Server data (lists, documents, quizzes)	TanStack Query cache	Short cache for course lists; no cache for review queue and attempt results
Form fields and steps	Component state	Validate before sending; keep server messages next to fields
Current folder path	URL	Folder navigation is the route, so back button and shared links work
Quiz answers in progress	Component state	Lost on refresh; acceptable for the MVP

Decision needed: where to store the token

The docs define a bearer JWT with no refresh token. Keeping it in localStorage is simple and survives app restarts in the installed PWA, but it is readable by any script on the page. The frontend proposes localStorage plus strict input escaping (React escapes by default) and no third-party scripts, and asks the security reviewer to confirm.

Every data screen has four states

- **Loading:** skeleton blocks, not spinners alone, so layout does not jump on slow networks.
- **Empty:** plain explanation and a next action (for example "No papers yet. Upload one.").
- **Error:** message from the API plus a Retry button.
- **Success:** the data, with pagination when the list has more than one page.

Document status labels in the UI

Status	Shown to	Badge
Under Review	Uploader and reviewers only	Amber
Verified	Everyone	Green, with the stamp ID
Rejected	Uploader and reviewers only	Grey
Revoked	Anyone with the verification link; not listed in the library	Red

Status names come from the PRD, SRS and the verification enum in the YAML. The concept paper also names a Removed state; the UI treats removed content as not found.

10. PWA and low-bandwidth plan

The course requires the PWA to be installable and phone-friendly from day one, not added at the end.

PWA requirements

Item	Plan
Manifest	name LetLearnTogether, short_name LetLearn, start_url /, display standalone, theme color blue (#1550B0), background white, icons 192 and 512 px
Service worker	Precache the app shell (HTML, JS, CSS, icons). Runtime caching with stale-while-revalidate for course lists only.
Never cache	Auth responses, tokens, uploaded files, review queue, quiz attempts
Offline behaviour	Show a clear offline banner and the cached shell. Full offline mode is a stretch goal, not part of the MVP.
Updates	Prompt the user to refresh when a new version is ready
Install	HTTPS only. Show an "Install app" prompt after login on supported browsers; the user manual covers iPhone and Android steps.

Low-bandwidth rules

- Split code by route so the first load only downloads the home and login code.
- Initial JavaScript target: under 200 KB gzipped. This is a team target, not a course rule; the build report checks it.
- System font stack, no web-font downloads; icons as inline SVG.
- Paginate every list (20 per page). Lazy-load images and document previews.
- Compress photos before upload in the browser. Resumable upload is a course goal; for the MVP, show progress and a Retry button.
- Test every screen with Chrome throttled to Slow 3G and on a real mid-range phone.

Checks before each demo

Check	Tool	Target
Installable	Lighthouse PWA audit, real phone	Install prompt works on Android; Add to Home Screen on iPhone
Speed on slow network	Chrome DevTools Slow 3G	Library list usable in a few seconds; skeletons appear immediately
Bundle size	Build output report	Under the 200 KB target
Accessibility	Lighthouse and manual keyboard pass	No blocking issues on core screens

11. Design system and accessibility

The interface uses a blue and white palette. Colors are CSS variables in styles/tokens.css so a change is made once.

Colors

Token	Value	Use
--blue-900	#0B2E6B	Headers, primary text on light blue, dark panels
--blue-700	#1550B0	Primary buttons, links, active states
--blue-500	#2F80ED	Focus rings, highlights (not for small text)
--blue-100	#E6F0FF	Card and panel backgrounds
--white	#FFFFFF	Page background
--text	#0F172A	Body text
--status-verified	#1E8E3E	Verified badge
--status-review	#B26A00	Under Review badge
--status-revoked	#C62828	Revoked badge and errors

Status is never shown by color alone: every badge also carries a text label.

Type scale (system font stack)

Style	Size	Weight	Use
Heading 1	24 px	Bold	Screen titles
Heading 2	20 px	Bold	Section titles
Heading 3	18 px	Semibold	Card titles, folder names
Body	16 px	Regular	All reading text and form fields (16 px also stops iOS zooming into inputs)
Small	14 px	Regular	Metadata such as year, semester, upload date
Caption	12 px	Regular	Badges and helper text only

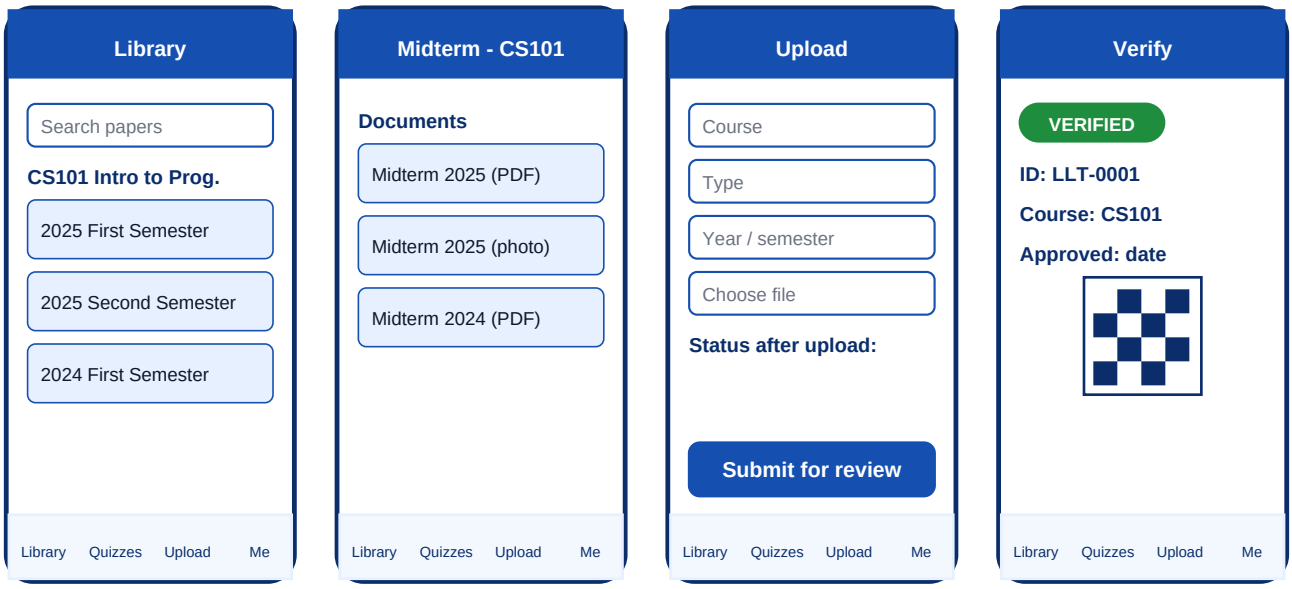
Layout and touch

- Single column on phones; content max width about 720 px on larger screens.
- Minimum touch target 44 by 44 px; 8 px spacing grid.
- Bottom navigation on phones (Library, Quizzes, Upload, My area); top navigation on wide screens.

Accessibility checklist

- Text contrast at least 4.5:1 against its background.
- Every input has a visible label; errors are announced next to the field.
- Full keyboard use with a visible focus ring; semantic HTML for headings, lists, buttons and links.
- Alternative text for the QR code and stamp images.
- Plain language and short sentences, since the course lists simple navigation as a goal.

12. Mobile wireframes (low fidelity)



Left to right: library folder level, document list at the type level, upload form, public verification page. The stamp ID and layout are examples; the real ID format is decided by the backend (Quizzes & Stamps owner).

Wireframe backlog (screens still to sketch)

Screen	Key content
Login / Register	Email, password, clear error text, link between the two
Document reader	Inline view or download button, status badge, report link
My uploads	List with status badges and reviewer note if rejected
Review queue (lecturer)	Pending items, preview, Approve and Reject buttons
Quiz list and take quiz	One question per screen, progress indicator, Submit
Quiz result	Score, per-question review (needs API support)
Attempt history	Quiz, date, score
Moderation reports	Report reason, document link, Revoke or Remove
Admin users and courses	Table with role change and lecturer assignment

The course accepts hand-drawn wireframes that are photographed, so the team can sketch these on paper and attach the photos.

13. Delivery plan and responsibilities

Dates follow the course concept paper. The PRD schedule is a week behind it (see section 15), so the course dates are used here.

Week / due	Frontend deliverable	Done when
Week 2 - Tue 27 Oct	Architecture diagram, wireframes, PWA shell, this pack, defence of the frontend section at design review	Pack submitted; shell runs locally
Week 3 - Tue 3 Nov	Auth screens, library folders, search, document reader and download, upload with status, installable PWA on the domain	Student can register, upload, find and download on a phone (AC-01, 02, 05, 07)
Week 4 - Tue 10 Nov	Review queue, stamp and verification page, quiz list, take quiz, score and history, role-based dashboards, integration	AC-03, 04, 06 pass on the live site
Week 5 - Tue 17 Nov	Polish, accessibility and low-bandwidth checks, user manual with screenshots (site and PDF), oral defence	Manual tested by QA step by step

Team responsibilities (from PRD v3.0)

Member	Role	Tasks
Prince M Gbolon	Frontend Team Lead	PWA setup, routing, state management, code reviews
Kim F Karmbor	UI Components	Layout, navigation, reusable buttons and forms, mobile styling
Estella Tugbe	Student Library	Upload screen, search and filter screen, document download UI
Jerome Allison	Dashboards and Quizzes	User dashboards, quiz-taking UI, verification page UI

Working rules

- No direct commits to main; every change goes through a pull request reviewed by another member.
- State in each pull request where AI helped, which API paths it uses, and how it was tested.
- Never commit passwords, API keys, real student data or real exam files. Use sample data only.
- Anyone at risk of missing a deadline tells the lecturer in writing before the deadline, with a recovery plan.
- Extras (ratings, offline mode, notifications) start only after the required screens work, and never on the live site until stable.

14. Testing and definition of done

Type	Tool	What it covers
Component tests	Vitest + React Testing Library	Forms, badges, pagination, error banner, role guards
API mocks	MSW from the OpenAPI contract	Building and testing screens before the backend is ready
Integration	Against local backend (localhost:8000)	Login, upload, review, download, verify, quiz scoring
Device checks	Real phones, Chrome device mode	Small screens, touch targets, install flow
Performance and PWA	Lighthouse, Slow 3G throttling	Section 10 targets

Frontend share of the PRD acceptance criteria

ID	Criterion	Frontend evidence
AC-01	Register and login; blocked from protected routes	Guards redirect to login; test for 401 handling
AC-02	Upload with metadata and see review status	Upload form and My uploads screen
AC-03	Lecturer approves or rejects; action logged	Review queue screens (logging is backend)
AC-04	Approved document has QR stamp; public page shows status	Stamp display and /verify/:token page
AC-05	Search, read and download on a mobile device	Folder browse, search, reader, download
AC-06	Take a quiz, see score and history	Quiz screens and history list
AC-07	Installable PWA over HTTPS on the domain	Manifest, service worker, Lighthouse report

Pull request checklist

- Works at phone width and on a throttled connection.
- Loading, empty, error and success states exist.
- Uses the API client; no hard-coded URLs.
- No secrets or real data; tests added or updated; AI use noted.

15. Contract alignment and open questions

These items were found by comparing the PRD, SRS, backend docs and the draft YAML. Each row says what the frontend assumes until the owner decides.

#	Issue	Frontend assumption until resolved	Owner
1	Draft YAML has about 8 operations; backend docs list more and call the YAML authoritative	Build against the docs; send the current YAML	Backend lead
2	YAML register accepts a role field including Admin; docs say public registration creates Students only	Never send role	Backend (Identity)
3	Attempt route: /attempt (YAML) vs /attempts (docs)	Use /attempts; one-line change	Backend lead
4	Revoke: review action (YAML) vs separate route; docs say Lecturers cannot revoke	Isolate in api/documents.js; hide Revoke from Lecturers	Backend (Docs)
5	No endpoint listed for pending documents of a lecturer's courses	Assume GET /documents with a status filter	Backend (Docs)
6	Document list lacks page, page_size, keyword, status, lecturer filters; no response schema	Send page and page_size; ask for field list	Backend (Docs)
7	Fourth folder level (lecturer) in PRD, missing from SRS route and parameters	Show lecturer as a filter on the type page	Product Owner
8	Courses table keeps course_code unique and also stores year and semester, so the same course cannot appear in two years	Frontend builds levels 2-3 from documents; ask DB to fix	Database
9	Documents table lacks uploader, title, reviewer and approval date, which the screens need	Request these fields in the response	Database / Backend
10	Quiz tables have no questions or options table; attempts do not store answers	Request per-question results in the attempt response	Database / Backend
11	Quiz answer review (course paper) needs correct answers after submission	Build result screen with score first; add review when API supports it	Backend (Quizzes)
12	What URL does the QR code encode? It must open the frontend /verify/:token page	Assume https://domain/verify/{token}	Backend (Quizzes) / OPS
13	Verification response lacks document ID (SRS FR-VER-02 requires it)	Show ID when provided	Backend (Quizzes)
14	Upload form lacks year and semester metadata required by the course paper	Add fields when the API accepts them	Backend / Database
15	Reading and downloading: public (backend docs) vs Student (SRS)	Public reading	Product Owner
16	No refresh token; token lifetime unknown	On 401, go to login and return to the same page	Backend (Identity)
17	Report submission path not stated	Placeholder in api/reports.js	Backend (Docs)
18	PRD sprint dates are a week behind the course schedule and have four sprints against the course's five weeks	Use course dates	Product Owner / Lecturer
19	CORS and production API URL for the real domain	Use env variable; request production origin	OPS
20	Token storage choice (section 9)	localStorage with strict escaping	Security

Approvals

Role	Name	Signature	Date
Frontend Team Lead			
Backend Team Lead			
Product Owner			
Supervisor			